

6809 Forth — Open Items Checklist (Part 1 of 2: Port & Core Bug Fixes)

Everything below was explicitly flagged during the build as incomplete, unverified, or deliberately deferred. Nothing here is a surprise — each item was named at the point it came up. This is a consolidated list to work from, not a new set of findings. Regenerated to reflect fixes and design decisions made since the original version.

This is part 1 of 2, covering the initial 6809 port and its core bug fixes, up to (but not including) the point the assembly-level unit test framework was introduced. See part 2 for the unit test framework itself and every glossary-section test group built on it since.

Known bugs — resolved since the original checklist

- **DUMP 's partial-final-line bug** — fixed via `DUVALID` tracking (`25_tools_word_set.asm`). The ASCII column no longer reads past the valid bytes on a short final line.
- **SUBSTITUTE** — completed. It now performs a real bounds-checked copy (prefix / replacement / suffix) via a shared `SUBCOPY` helper, reusing `SEARCHW` for the match. Still scoped to a single registered name/value pair, not a full table — see “Open design questions” below; that scope limit is a deliberate simplification, not a bug.
- **VALUE 's PFA** — moved from `CODEHERE` (immutable space) to `VARHERE` (mutable space), matching `VARIABLE` 's pattern, so every `TO` -driven update now writes into the correct region instead of into code space. Not on the original checklist (found and fixed after this list was first generated); logged here for the record.
- **Every scratch/global cell now has a real `RMB` -assigned address in page zero** (`00_memory_map_and_globals.asm`), applied rather than left as a placeholder. Two real findings came out of doing this: the budget is **253 of 256 bytes used — only 3 bytes of headroom** in the GLOBALS page, and `SNEND` (documented in the original placeholder list) turned out to be genuinely dead — never read or written anywhere — and was dropped rather than given an address. Any future scratch cell added to this system will need to fit in that remaining 3 bytes or the page-zero fast-addressing property (`DP = $00`) stops covering it.
- **The ROM base dictionary is now real** (`27_forth_dictionary.asm`) — every primitive with actual code got a real header, chained via `LINK` , `CFA` pointing straight at its code

label. `ABORTHDR` 's `LINK` field, a placeholder `0` since it was first built, is now resolved to the chain's newest entry. Building this surfaced two real findings, not just mechanical work: the original 1024-byte `BASEDICT` could not hold the header table (ultimately 1954 bytes needed, once `DOES>` was added — see below), so it was resized to 2048 bytes, taking the space from `BASECODE` (now 6080 bytes, was 7104) — a resize based purely on the header-table budget, not on any actual measurement of `BASECODE` 's assembled size, which has never been checked with a real 6809 assembler. Generating this table also surfaced that `DOES>` **had no corresponding code anywhere in the file** — resolved in a follow-up pass, see below.

- **DOES> — resolved.** `SETDOES` (the patching runtime) was added beside `DODOES` / `DOESRT0` , and `DOES>` itself (code label `DOESGT` , since a literal `">"` is not a valid 6809 assembler label) was added right after `CREATE` . `DOESBEH` was added to the `GLOBALS` layout to support it, using 2 of the page's last 3 free bytes — only 1 byte of headroom now remains in page zero. `DOES>` also has a real dictionary header now (`H_DOESGT` , the chain's newest entry); `ABORTHDR` 's `LINK` was updated again to point at it.
- **Four internal loop-label names were reused across unrelated routines:** `FLOOP` , `UDLOOP` , `DRPOS` , `CMLOOP` . Resolved — each pair renamed to a distinct name scoped to its own routine: `FIND` 's `FLOOP` → `FFLOOP` ; `FILLW` 's `FLOOP` → `FILLOOP` ; `UDIV16` 's `UDLOOP` → `UD16LOOP` ; `UDDIGIT` 's `UDLOOP` → `UDDLOOP` ; `DIVCOMMON` 's `DRPOS` → `DCRPOS` ; `DDOTR` 's `DRPOS` → `DDRPOS` ; `CMOVEW` 's `CMLOOP` → `CMVLOOP` ; `COMPAREW` 's `CMLOOP` → `CMPLOOP` . Verified zero duplicate labels and zero stray references to any of the old names remain anywhere in the file.
- **Hardware (RTS) serial handshaking — implemented.** `IRQH` 's receive path and `KEY` now toggle RTS based on the input ring's fill level (`INFILL` , `RTSCHECKHI` , `RTSCHECKLO` against `INHIWATER` =48/ `INLOWATER` =16, with hysteresis between them) via a new `CR_RTSHI` control byte and `RTSSTATE` flag — the last free byte in the `GLOBALS` page, which is now fully packed at 256/256. CTS (the other handshaking direction) needed no firmware logic at all: confirmed against the 6850 datasheet that TDRE is automatically inhibited while CTS is deasserted, so this system's existing TDRE-gated transmit logic already respects it. A real race was identified and closed: mainline code (`KEY` , via `RTSCHECKLO`) and the ISR (`IRQH` 's `TXOFF`) can both decide to write the control register, so `RTSCHECKLO` masks IRQ around its critical section and `TXOFF` checks `RTSSTATE` before writing. Software (XON/XOFF) handshaking remains unimplemented — see below.
- **TRUE and FALSE — implemented.** Surfaced while organizing the ANS test suite: neither existed as an actual dictionary word, only as the internal `TRUEV` / `FALSEV` assembler constants. Added as `CONSTANT TRUE -1` / `CONSTANT FALSE 0` (`TRUEBODY` /

`FALSEBODY` , section 26; headers `H_TRUE / H_FALSE` , chain's two newest entries, section 27) — the first `CONSTANT` -pattern ROM-resident words in this system. Every other ROM word's CFA is a plain code label; a `CONSTANT` 's CFA is the `DODOES` -trampoline pattern instead, built here with fixed, assemble-time addresses rather than a `CODEHERE` snapshot, since there is no interactive `CREATE / CONSTANT` phase for ROM content.

- **One genuine 6309-only instruction (`TSTD`) was in use — fixed.** Found by auditing every mnemonic in the file against the real 6809 instruction set, rather than trusting the Build Instructions section's earlier claim that no 6309 extensions were used (that claim was wrong until this fix). `TRYNUM` (section 9) used `TSTD` with no operand to test whether `D` was zero after `PULU D — PULU / PULS` don't affect condition codes on genuine 6809 hardware, unlike a load, so this relied on a 6309-only convenience instruction. Replaced with `CPMD #0` , a real 6809 instruction with identical behavior for this purpose. A full mnemonic audit after the fix found zero remaining non-6809 instructions anywhere in the file.
- **`BASELATEST` was an undefined symbol — a real assembly-breaking bug, not a placeholder.** `COLD` (section 4) has referenced `LDD #BASELATEST` to initialize `LATEST` since this file's earliest version, and the SECTION 27 header comment has long asserted "`BASELATEST` remains `QUITHDR`" - but no `EQU` or label actually named `BASELATEST` existed anywhere in the file. This would have failed to assemble outright. Fixed by adding `BASELATEST EQU QUITHDR` directly after `QUITHDR` 's own definition (section 26), matching what the comment always said it should equal. Verified: exactly one definition, no duplicate symbols, and `COLD` 's forward reference to it resolves under standard two-pass assembly.
- **`JSR CR` (bare, undefined) appeared at five call sites — a real assembly-breaking bug, not a naming inconsistency.** The actual routine has always been labeled `CRW` (section 22), following this file's own convention of giving short/symbolic ANS names a distinct code label. `ABORT` , `QUIT` 's error-report path, `QUIT` 's `ok` -prompt path, `WORDS` (`WWDONE`), and `DUMP` (`DULEND`) all called the bare, undefined `CR` instead. Fixed at each site to `JSR CRW` , preserving each call site's original spacing exactly. Verified: `CRW` defined exactly once, referenced 7 times total, zero remaining bare- `CR` instruction operands anywhere in the file (the two harmless bare "`CR`" occurrences that remain are a section-title comment and the dictionary header's `FCC "CR"` name string, neither of which is a bug).
- **Two ALU instructions used invalid register-to-register syntax (`ADDA B , EORA B`) — the 6809 has no such addressing mode, so the assembler read `B` as an undefined direct-page symbol.** Found in the single-cell multiply routine (`ADDA B` , twice) and `DOPLUSTEST` 's sign comparison for `+LOOP` (`EORA B`). Fixed with the standard 6809

idiom for combining two registers: `PSHS B` followed by `ADDA ,S+ / EORA ,S+` (push B, then operate through the auto-incrementing stack-indexed operand). Verified: a full sweep of every ALU instruction (`ADDA / ADDB / SUBA / SUBB / ANDA / ANDB / ORA / ORB / EORA / EORB / CMPA / CMPB / ADCA / ADCB / SBCA / SBCB / BITA / BITB`) against a bare `A / B` operand found zero remaining instances; every other bare `B` in the file (in `STA / LDA / LEAX ... ,X` and `PSHS B`) is genuine, valid 6809 accumulator-offset indexed addressing or register-list syntax.

- Nine scratch symbols (`MRESULT` , `MVCNT` , `MVDST` , `MVSRC` , `FILLCHR` , `FILLCNT` , `FILLADDR` , `HSLEN` , `HSADDR`) were used throughout `MOVE / CMOVE / CMOVE>` , `FILL` , `HOLDS` , and the single-cell multiply routine but never declared anywhere — a real, assembly-breaking gap.** Auditing every one of the 142 already-declared GLOBALS cells for actual use found zero dead cells to reclaim this time (unlike `SNEND` earlier), and the GLOBALS page was independently confirmed at exactly 256/256 bytes with no headroom. Since `MOVE` -family, `FILL` , `HOLDS` , and the multiply routine never call each other or run concurrently in this single-threaded interpreter, they now share physical storage: three new cells (`MVCNT` , `MVDST` , `MVSRC`) hold the real storage, and `FILLCNT / HSLEN` alias `MVCNT` , `FILLADDR / HSADDR` alias `MVDST` , and `MRESULT / FILLCHR` alias `MVSRC` , all via `EQU` . These three new cells could not fit in the full GLOBALS page, so they live at `$0100` , carved from the front of `USER0` (shrunk from 128 to 122 bytes, now starting at `$0106`) — a real, documented tradeoff: these three cells use ordinary extended addressing, not direct-page, including inside `CMOVEW` 's per-byte copy loop.
- `JSR SPACE` (bare, undefined) — same class of bug as `CR` , fixed the same way.** `WORDS` called the bare, undefined `SPACE` instead of the actual routine label `SPACEW` (section 22). Fixed to `JSR SPACEW` . Checked `SPACES` for the same pattern (none found — `SPACESW` correctly calls `EMIT` directly) and swept the file for any other bare `SPACE` instruction reference (none remain; the two harmless bare "SPACE" occurrences left are a section-title comment and the dictionary header's `FCC "SPACE"` name string).
- Three short branches (`BHS` , `BEQ` , `BNE`) overflowed the 8-bit short-branch range (± 127 bytes) — a real assembler error, not a style issue.** `SUBCOPY` 's overflow check (`BHS SUBOVERFLOW` , ~87 source lines to target), `DUMPW` 's per-line loop test (`BEQ DUDONE` , ~76 lines), and `DUMPW` 's loop-back (`BNE DULINE` , ~76 lines) all spanned too much code for an 8-bit displacement. Fixed by converting each to its long-branch equivalent (`LBHS / LBEQ / LBNE`), which uses a 16-bit displacement (± 32767 bytes) - functionally identical, just not range-limited.
- RESOLVED via the real assembler listing (`forth6809_1st.txt` , uploaded this session): all ten previously-flagged short branches confirmed genuinely safe, not just "didn't error."** Checked each by direct address arithmetic against the listing's own

computed addresses and encoded opcode bytes, rather than relying on “it assembled without an error” alone: QLOOP 's three branches (BRA at \$DF92 → \$DF43 , offset -81 / \$AF ; BNE at \$DF99 → \$DF43 , offset -88 / \$A8 ; BRA at \$DFA8 → \$DF43 , offset -103 / \$99), FFLOOP / NOTFOUND (BEQ at \$E486 → \$E4DF , offset 87 / \$57 ; BRA at \$E4DD → \$E484 , offset -91 / \$A5), ALOOP / ADONE (BEQ at \$E658 → \$E6A9 , offset 79 / \$4F , and the remaining ALOOP branches, all 2-byte-encoded), and UELOOP / UEDONE (BEQ at \$F87A → \$F891 , offset 21 / \$15 ; BRA at \$F88F → \$F878 , offset -25 / \$E7). Every offset decoded from the listing's own byte column matches the computed target-minus-(address+2) distance exactly, and all are comfortably within the ±127-byte signed range - none needed conversion to LBxx .

- Ten more short branches have a large (41-51 source line) span to their target and were not individually confirmed safe.** Found by a heuristic sweep (line-count as a rough proxy for byte distance, since no real assembler is available in this environment to get an exact count) after fixing the three confirmed overflows above, all of which spanned 76+ lines - these ten are meaningfully shorter but not verified within range: QUERY 's QLOOP (three branches, lines 579/582/589), FIND 's NOTFOUND / FFLOOP (lines 1210/1256), ACCEPT 's ALOOP / ADONE (lines 1501/1462), UNESCAPE 's UEDONE / UELOOP (lines 3888/3931), and EMPTY (line 1153). None of these were reported as failing and none have been changed; if a real assembler flags any of them, the fix is the same: convert to the matching LBxx long-branch form. **RESOLVED - see the entry immediately above, confirmed via the real assembler listing.**
- ORG BASECODE was missing entirely — a fundamental placement bug, not a cosmetic gap.** VECTORS (\$FFF0) and INIT (\$FFC0) both had their own ORG , but SECTION 3 (IRQH) — and every routine through SECTION 26 , i.e. nearly the entire interpreter — had no ORG placing it in BASECODE at all. Without it, this code would have continued growing from wherever INIT 's WARM message left the location counter, inside INIT 's own 48-byte \$FFC0-\$FFEF budget, overflowing directly into VECTORS rather than landing in BASECODE (\$E800-\$FFBF) anywhere. Fixed by adding ORG BASECODE immediately before SECTION 3 begins.
- Superseded by a later, more precise count** (see the “Follow-up” entry below with the 7984-byte instruction-by- instruction result) - the 8660-byte figure here used a cruder heuristic and a 6080-byte budget that's since changed (BASECODE is now 8110 bytes nominal, after the BASECODE / BASEDICT 30-byte shift). Kept as history, not deleted, since it was a genuine finding at the time - just not the current best estimate. Original text: **Adding the missing ORG makes BASECODE 's byte budget concretely checkable for the first time — and a rough estimate suggests it may not fit.** This was previously flagged only as “unverified” (no real assembler has ever been run against this file); with a real ORG boundary now in place, a heuristic per-instruction byte count

(inherent/direct-page/ indexed/immediate/extended opcode sizes, correctly distinguishing direct-page GLOBALS operands from extended ones) over every instruction from SECTION 3 through SECTION 26 estimated roughly 8660 bytes against a 6080-byte budget — about 2580 bytes over. The underlying question (does BASECODE actually fit, confirmed by a real assembler) is still genuinely open - see the later, more precise follow-up entry.

- **USER0 and USER1 (250 bytes of unallocated reserve space, never read or written by any code) were deleted, and the region from MVSCRATCH through APPDICT was made fully contiguous.** SERBUF, INBUF, OUTBUF, TIBBUF, WORDBUF, and SIBUF all shifted down to sit immediately after MVSCRATCH (\$0106) with no gaps between any of them. This also closed a separate, pre-existing 11-byte gap between WORDBUF and SIBUF (\$02F5-\$02FF) that had nothing to do with USER0 / USER1 but was caught while verifying the region was genuinely contiguous end to end. APPDICT was extended downward to close the resulting gap too (new start \$021B, was \$0320) — its end (\$6EFF) is unchanged, so this is a pure 261-byte gain in application dictionary space, not a resize of its budget. This was an interpretation of "contiguous," not something explicitly asked for beyond the 6 buffers themselves; flagged here in case a fixed APPDICT start was actually intended instead. Verified: zero duplicate symbols, zero remaining references to USER0 / USER1 anywhere in the code, and the full GLOBALS -> MVSCRATCH -> buffers-> APPDICT span checked programmatically for zero gaps.
- **APPVARS and APPDICT swapped positions - APPVARS now sits below APPDICT .** APPDICT moved from \$021B to \$031B (still ending at \$6FFF, directly below APPCODE); APPVARS moved from \$6F00 down to \$021B (same 256-byte size, now sitting directly above the buffers instead of directly below APPCODE). Checked before making the change: only two code references exist for either symbol (COLD initializing VARHERE / DPHERE to each region's start) and neither depends on their relative order, so the swap was safe. Verified: zero duplicate symbols, the whole region from GLOBALS through APPCODE 's start remains contiguous with zero gaps, dictionary chain unaffected (219 entries, since this change doesn't touch it at all).
- **APPVARS grown from 256 to 8000 bytes, taking the space directly from APPDICT .** APPVARS now spans \$021B-\$215A (start address unchanged); APPDICT shrank by the same 7744 bytes to \$215B-\$6FFF (end unchanged, still directly below APPCODE), giving 20133 bytes of application dictionary space, down from 27877. Checked before making the change: still only two code references to either symbol (COLD 's VARHERE / DPHERE initialization), neither size-dependent. Verified: zero duplicate symbols, APPVARS size is exactly 8000 bytes, the whole region stays contiguous end to end, dictionary chain unaffected.

- ACIA (the 256-byte memory-mapped I/O block) renamed to `INOUT` , and the actual 6850 ACIA chip's registers moved to `INOUT+8` instead of the block's base.** The block still spans the same 256 bytes (`$DF00-$DFFF`); it's now modeled as a general I/O region with the ACIA chip occupying one small part of it (`ACIACR / ACIASR = INOUT+8 = $DF08` , and the data register `ACIADR` at `$DF09`), leaving `INOUT+0 .. INOUT+7` free for other memory-mapped devices sharing the block. Doing this rename surfaced a real, previously-hidden bug: `COLDSTRT` 's ACIA reset sequence used `STA ACIA` (the block's base) instead of `STA ACIACR` (the control register) in two places - this only ever worked because `ACIA` and `ACIACR` happened to be the same address in the old scheme. Once `ACIACR` moved to `INOUT+8` , writing to the bare block base would have silently stopped resetting the ACIA at all. Fixed both to `STA ACIACR` . Verified: `INOUT` defined once, the bare `ACIA` symbol (at the time) no longer existed anywhere, and every other ACIA register access in the file already used the correctly-named register symbols rather than the bare block name.
- ACIA reintroduced as an explicit base symbol (`ACIA EQU INOUT+8`), with `ACIACR / ACIASR` now defined relative to it (`EQU ACIA`) rather than directly to `INOUT+8` .** The data register was briefly renamed to `ACIRDR` in the same pass, then reverted back to `ACIADR` immediately afterward once flagged as a typo - both code sites (`IRQH` 's receive and transmit paths) were updated each time the name changed. Verified: each of `ACIA / ACIACR / ACIASR / ACIADR` defined exactly once, resolving to the expected values (`ACIA = INOUT+8` , `ACIACR / ACIASR = ACIA` , `ACIADR = ACIA+1`), zero remaining references to `ACIRDR` anywhere, zero duplicate symbols, dictionary chain unaffected.
- `INITEND / INITSIZE` landmark constants added at the true end of the `INIT` block** (`INITEND EQU *` , `INITSIZE EQU *-COLDSTRT`), right after `WARMMSSL` . The request referenced a `COLDSTART` label, which doesn't exist in this file - used the actual existing label `COLDSTRT` instead of introducing a new undefined symbol.
- CORRECTED: the manual 78-byte count below was wrong - a real assembler run reports `INITCODE` 's actual size as 71 bytes (`$47`).** This entry's own last line said to trust `INITSIZE` once the file was actually assembled rather than this manual count - that's now happened, and the manual count was off by 7 bytes. See the later entry recording the correction in full; kept here as history, not rewritten. Original text:
`INITEND` 's comment now states the invariant explicitly ("value should match vector `ORG`") - and checking it confirms it currently holds. The precise instruction-by-instruction byte count from when the `VECTORS` collision was first found (78 bytes exactly, `COLDSTRT` through `WARMMSSL`) gives `INITCODE` start (`$FFA2`) + 78 = `$FFF0` , exactly matching `VECTORS` ' `ORG` . Zero gap, zero overlap - unlike the still-open

`INITCODE / BASECODE` overlap at the other end of this same region, which remains a real problem. Same caveat as always: this is a manual count, not a real assembler's output, so `INITSIZE` (computed automatically at assembly time) is the figure to trust once this file is actually assembled.

- **`BASECODESTRT / BASECODEEND / BASECODESIZE` and `BASEDICTSTRT / BASEDICTEND / BASEDICTSIZE` landmark constants added, matching the `INITEND / INITSIZE` pattern - and checking the two stated invariants gives one clean confirmation and one confirmation of an already-known problem.**

`BASEDICTEND` (`$D85D` + the exact 1973-byte dictionary content = `$E012`) matches `ORG BASECODE` (`$E012`) precisely - this boundary is genuinely correct, not just close.

`BASECODEEND`, using the same rough heuristic estimate flagged much earlier (~8660 bytes, never confirmed with a real assembler) starting from `BASECODESTRT` (`$E012`), lands around `$101E6` - not only failing to match `ORG INITCODE` (`$FFA2`) as the comment expects, but overflowing past `$FFFF` entirely on that estimate. The exact amount of room actually available between `BASECODESTRT` and `INITCODE` (`$FFA2` - `$E012` = 8080 bytes) is itself less than the ~8660-byte estimate, meaning even filling every available byte up to `INITCODE` with zero gap would still likely fall short. This is the same underlying problem already tracked elsewhere (`BASECODE` possibly not fitting its budget, and separately the `INITCODE / BASECODE` overlap), now independently reachable via these new landmarks once the file is actually assembled, rather than a new, different issue.

- **`SUPERSEDED` - see the real-listing entry below with the definitive 8304-byte figure.** Follow-up: a real instruction-by- instruction count (not the rough heuristic above) puts `BASECODE` 's actual size at 7984 bytes (`$1F30`), 96 bytes (1.2%) short of a target assembler- listing value of `$1F90` (8080 bytes) given for comparison. Built a mnemonic-and-addressing-mode-aware counter distinguishing inherent/immediate/direct/extended/indexed forms, correct prefix requirements (`$10 / $11` for `LDY / STY / LDS / STS / CMPD / CMPY / CMPU / CMPS`), and true direct-page symbols (only `$0000 - $00FF`, matching `DP`) - a meaningfully more rigorous pass than the original ~8660-byte estimate. Every one of the 3751 instructions in the region was classified (zero "unrecognized" remaining after fixing early misses like `LSLB / LSLA` as `ASLB / ASLA` synonyms). Checked for likely sources of the remaining 96-byte gap before accepting it: zero indexed operands use an offset outside the 5-bit range that would need extra encoding bytes (all 125 numeric-offset operands found are small, e.g. `1,X / -1,Y`), and the two apparent "indirect []" matches were a false positive (a section-title comment, not real indirect addressing). The target value `$1F90` is notable in its own right: `BASECODE + $1F90 = $FFA2` exactly, meaning if the real assembled size actually matched it, `BASECODE` and `INITCODE` would be perfectly contiguous with zero

gap and zero overlap - which would also resolve the separate, still-open

`INITCODE / BASECODE` overlap noted elsewhere. My count doesn't confirm that, though it's close (1.2% off). This is still not a real assembler's output - the standing caveat throughout this file - and the gap could come from encoding edge cases a manual model can approximate but not guarantee against.

- **DEFINITIVE, real figures now available from `forth6809_1st.txt` (a real assembler listing, uploaded and confirmed to match the current source exactly -**

`BASECODE=$DEEA` , `UNITTESTS=1` , and this session's `ENVTABLE` additions are all present in it). `BASECODESIZE` = 8304 bytes (`$2070`) - the manual instruction-by-

instruction count above (7984 bytes) undershot this by 320 bytes (about 4%), confirming that entry's own caveat ("the gap could come from encoding edge cases a manual model can approximate but not guarantee against") was warranted.

`BASEDICTSIZE` = 2027 bytes (`$07EB`). `INITSIZE` = 71 bytes (`$47`), matching the "told directly" figure elsewhere in this file exactly. `VECTORSIZE` = 16 bytes, as expected. Two genuinely new, previously-unverifiable facts this listing settles outright: `BASEDICTEND` equals `BASECODE` 's start exactly (`$DEEA`) - zero gap, zero overlap, fully contiguous - and `INITEND` equals `VECTORS` 's start exactly (`$FFF0`) - also zero gap. The one remaining space (`BASECODEEND` `$FF5A` to `INITCODE` `$FFA9` , 79 bytes) is not an unaccounted gap: the listing shows the `FILL $FF,INITCODE-BASEND` directive actually executing, producing real `$FF` bytes across exactly that span - resolving the long-standing " `FILL` 's status as a real LWASM directive remains unconfirmed" uncertainty at the same time (see below). Total real ROM content (`VECTORS` + `INITCODE` + `BASECODE` + `BASEDICT` = 16+71+8304+2027 = 10418 bytes) and the resulting unused-ROM figure (5710 bytes, ~35%) were already updated in the documentation's Memory Map section using these same real numbers.

- **`FILL` 's status as a real, supported LWASM directive - previously flagged as unconfirmed in multiple entries throughout this file - is now conclusively resolved: it is real.** Both uses in the source (the `ROM: block`, `FILL $FF,BASEDICT-ROM` , and the `BASEND: block`, `FILL $FF,INITCODE-BASEND`) appear in the real listing with actual `$FF` - filled bytes shown in the output column, at the expected addresses. Cross-checked the two fill sizes against each other as a consistency check, not just individually: `ROM: block` fills `$C100` to `BASEDICT` (`$D6FF`), 5631 bytes; `BASEND: block` fills `$FF5A` to `$FFA9` , 79 bytes. The two sum to exactly 5710 - matching the "5710 bytes unused" figure already published in the Memory Map documentation precisely, not approximately, confirming that figure was internally consistent with the real fill directives all along.
- **`ENVTABLE` completeness - the item below listing `/HOLD` , `/PAD` , `MAX-D` , `MAX-UD` , `WORDLISTS` , `FLOORED` as missing is now fully resolved.** All six were added and MAME-

confirmed correct over several turns this session (`/HOLD =34`, `/PAD =84`, `MAX-D = $7FFFFFFF`, `MAX-UD = $FFFFFFFF`, `WORDLISTS =correctly unrecognized/false` since Search-Order isn't implemented, `FLOORED =0/false` after tracing this system's actual division convention against `SM/REM / FM/MOD`). `MAX-CHAR`, `RETURN-STACK-CELLS`, and `STACK-CELLS` were also added and confirmed beyond what this item originally flagged. The double-cell dispatch gap this item specifically called out ("need the dispatcher extended to push *two* cells... current `ENVQUERY` only handles one") was resolved with a genuine dispatcher extension (`ENVTABLE2 / ENV2LOOP / ENV2FOUND`), not a workaround. `ENVIRONMENT?` is complete.

- ROMSTRT / ROMEND added as the outer ROM boundary (`$C100 / VECTORS-1`), and INITCODE / BASECODE / BASEDICT verified contained within it - all three pass.** `ROMEND` was initially defined as `$10000` ("one past `VECTORS` 's end," a symbolic value that doesn't fit as a real 16-bit address), then corrected to `VECTORS-1` (`$FFEF`, one before `VECTORS` 's start) - a genuine 16-bit address usable directly in comparisons or as a memory operand, not just symbolic arithmetic. Re-checked after the correction rather than assumed still valid: `INITCODE` (`$FFA2-$FFEF`), `BASECODE` (`$E012-$FFBF`), and `BASEDICT` (`$D85D-$E011`) are all still genuinely within `ROMSTRT..ROMEND - INITCODE` 's own end now lands exactly on the new boundary, a tighter and more meaningful check than before, not a violation. This containment check passes cleanly, independent of the other open problems below.
- ROMSTRT / ROMEND renamed to USROMSTRT / USROMEND, each with a short "Usable ROM start"/"Usable ROM end" comment.** Cosmetic, not functional - neither symbol was referenced by any code, only by nearby comments (three sites, all updated: the pair's own definitions and `INOUT` 's cross-reference). Verified: zero remaining bare `ROMSTRT / ROMEND` references anywhere, zero duplicate symbols, dictionary chain unaffected.
- VECTOREND / VECTORSIZE landmark constants added at the true end of the vector table, matching the INITEND / INITSIZE pattern - and checking the stated invariant confirms it exactly, not approximately.** Unlike `INITEND / BASECODEEND` (which needed a manual, approximate instruction-by-instruction byte count since real 6809 instructions have variable-length encoding), the vector table is pure `FDB` data with a fixed, unambiguous 2-byte width per entry - counting the 8 entries (`VRESV` through `VRESET`) gives exactly 16 bytes, matching the comment's stated `$10` expectation precisely. Verified: zero duplicate symbols, dictionary chain unaffected.
- BASECODESTRT removed as requested - but BASECODESIZE depended on it, so removing it alone would have left a genuine dangling undefined-symbol reference, the same bug class found and fixed several times earlier in this file.** Confirmed first

that `BASECODESTRT` was numerically identical to `BASECODE` itself (only comments sit between `ORG BASECODE` and where `BASECODESTRT` was defined, zero emitted bytes), then fixed `BASECODESIZE` to read `BASECODEEND-BASECODE` directly instead - matching the same pattern just applied to `INITSIZE / INITCODE`, not a new approach invented for this case. Verified: zero remaining `BASECODESTRT` references anywhere, `BASECODESIZE` resolves cleanly, zero duplicate symbols, dictionary chain unaffected.

- **`BASEDICTSTRT` removed the same way, for the same reason - `BASEDICTSIZE` depended on it too.** Same fix pattern as `BASECODESTRT` immediately above, applied to its counterpart: confirmed `BASEDICTSTRT` was numerically identical to `BASEDICT` (only `ORG BASEDICT` sits before it, zero emitted bytes), removed it, and repointed `BASEDICTSIZE` to `BASEDICTEND-BASEDICT` directly. Verified: zero remaining `BASEDICTSTRT` references anywhere, `BASEDICTSIZE` resolves cleanly, zero duplicate symbols, dictionary chain unaffected.
- **`INOUT` moved from `$DF00` to `$C000` (with `INOUTEND EQU INOUT+$FF` added immediately after), and every ACIA-related address verified between the two - also passes.** `ACIA / ACIACR / ACIASR ( $C008 )` and `ACIADR ( $C009 )` all fall within `INOUT - INOUTEND ( $C000 - $C0FF )`, checked numerically. Confirmed there is no other memory-mapped I/O device anywhere in this file besides the ACIA family - nothing else needed checking. This move has a real, positive side effect worth naming clearly: it resolves the `INOUT` portion of the three-way collision flagged when `BASEDICT` moved to `$D85D` two turns ago (`INOUT` no longer overlaps `BASEDICT`, since `$C0FF < $D85D`). The `DSTACK` and `RSTACK` portions of that same collision were untouched by this specific change, but have since been resolved separately - see below.
- **`INOUT` 's new collision with `APPCODE` is resolved, and so is the rest of the original `BASEDICT` collision - `RSTACK`, `DSTACK`, `CODETOP`, `APPCODE`, and `APPDICT` all moved down `$2000`.** `RSTACK ($BC00-$BEFF)` and `DSTACK ($B800-$BBFF)` no longer overlap `BASEDICT` at all - the three-way collision first found when `BASEDICT` moved to `$D85D` is now fully resolved (`INOUT` resolved it two turns ago, this resolves the remaining two thirds). `APPCODE` 's move (`$5000-$B7FF`) also separately resolves its overlap with `INOUT` from the previous entry, as a side effect of the same shift, not a second fix. Re-verified with a full pairwise sweep after the move, not assumed: the only overlap remaining from the four found last turn is the original, unrelated `INITCODE / BASECODE` one (30 B) - `INITCODE / BASECODE / BASEDICT / RSTACK / DSTACK / INOUT / APPCODE` are all now mutually clean.
- **`APPDICT` 's collision with `APPVARS` is now resolved - `APPVARSEND` changed from a static `APPVARS+8000` to `APPDICT-1`, making it self-derive from wherever `APPDICT` actually sits instead of a fixed size.** `APPVARSEND` is now `$1FFF` (was `$215B`), giving

APPVARS 7653 bytes of actual usable space (down from the originally-intended 8000 - the 347-byte reduction exactly matches the overlap this closes). APPVARS 's own EQU still starts at \$021B and its comment still cites the historical "8000 bytes" intent, now explicitly marked as the original intent rather than the current usable size. Functionally, APPVARS 's enforced range (\$021B-\$1FFF) no longer reaches APPDICT (\$2000-\$6EA4) at all - checked numerically, not assumed. This also makes the boundary self-correcting: if APPDICT moves again, APPVARSEND moves with it automatically, rather than needing another manual fix like the last several turns' worth of address changes required. VUNUSEDW 's own code is unchanged - it already computed against APPVARSEND (see below), so this fix took effect purely by changing what that symbol means. Verified: zero duplicate symbols, dictionary chain unaffected.

- DSTACK moved up \$200 (to \$BCFF), resolving both the CODETOP / DSTACK mismatch and the RSTACK / DSTACK gap from last turn in one move.** DSTACK 's new occupied bottom (\$B900 , from its \$BCFF top and 1024-byte size) now matches CODETOP (\$B900) exactly - APPCODE 's nominal ceiling no longer overstates safe growth room, and the 512-byte overlap that created between APPCODE 's nominal range and DSTACK 's true range is gone. As a bonus, not something separately requested: DSTACK 's new top (\$BCFF) is now exactly contiguous with RSTACK 's bottom (\$BD00), closing the 512-byte gap that had opened between them too. Verified with a full pairwise sweep after the move: exactly two overlaps remain in the current memory map - the still-open APPDICT / APPVARS one (347 B) and the original, unrelated INITCODE / BASECODE overlap (30 B) - down from three last turn.
- The VECTORS collision found by verifying INITEND is now resolved - INIT 's ORG moved from \$FFC0 to \$FFA0 .** Widened INIT from 48 to 80 bytes (\$FFA0-\$FFEF), comfortably covering the ~78 bytes COLDSTRT + WARM + WARMMSG actually needs (2 bytes of margin) - still an estimate from manual byte-counting, not a real assembler's output. INIT 's own ORG was also converted from a literal \$FFC0 to symbolic ORG INIT , matching BASECODE / BASEDICT 's convention, so the EQU and the actual placement can no longer drift apart the way BASECODE 's did before that was caught. One real, unresolved interaction worth naming: INIT 's new start (\$FFA0) is 32 bytes below the old documented BASECODE end (\$FFBF), tightening the still-open BASECODE overflow risk below by that much further - not a new problem in kind, since that risk was already estimated at roughly 2580 bytes over budget, dwarfing this additional 32-byte reduction, but worth tracking alongside it rather than treated as independent.
- RESOLVED (since this was written): the INITCODE / BASECODE overlap described below (30 bytes at the time) was fully closed several turns later** by shifting BASECODE and BASEDICT both down exactly 30 bytes - BASECODE 's nominal end now lands exactly one byte below INITCODE 's start, zero gap, zero overlap, confirmed by a

full pairwise sweep of the entire memory map. Kept as history below, not deleted.

Original text: **INIT moved again, from \$FFA0 to \$FFA2 (request contained a &FFA2 typo - used the correct \$ hex prefix instead).** INIT is now exactly 78 bytes (\$FFA2-\$FFEF), matching the COLDSTRT + WARM + WARMMSG byte-count estimate with zero margin (was 80 bytes, with 2 bytes of slack). This is a genuinely tighter fit than before, worth naming as a real tradeoff: any inaccuracy in the manual byte count, or any future addition to COLDSTRT / WARM, now has zero room to absorb. The overlap with BASECODE noted above is **not resolved by this change, only reduced** - from 32 bytes to 30 bytes (\$FFA2-\$FFBF). This was a real, pre-existing problem before this turn touched anything (\$FFA0 already overlapped BASECODE 's declared end by 32 bytes), not something newly introduced. Not fixed here, in either direction (shrinking BASECODE further or moving INIT above it instead of below): both are bigger architectural decisions than "change one EQU," and a real assembler run would settle the actual BASECODE byte count this depends on.

- **INIT renamed to INITCODE** (EQU and its ORG reference, both in the memory-map constants). The section-title comment ("SECTION 2: INIT CODE") and one historical narrative comment describing a past bug scenario (with the byte figures true at that time, not today) were deliberately left referring to "INIT" as plain English/history, not the symbol - renaming a symbol doesn't obligate rewriting prose that correctly describes what was true before the rename existed. Doing this rename surfaced that the documentation's "ROM Size Required" section (Section 2) had gone stale independent of the rename itself: it still asserted the four ROM regions were contiguous at exactly 8192 bytes, which stopped being true as soon as INIT moved to \$FFA2 last turn and started overlapping BASECODE . Rewritten to state the overlap plainly instead of the now-false claim. Verified: INITCODE defined exactly once, zero duplicate symbols, dictionary chain unaffected.
- **BASEDICT and BASECODE addresses applied exactly as requested, and the documented ROM requirement widened to a 16K part - but this surfaced a severe, unresolved collision, not a minor tradeoff.** BASEDICT moved from \$E000 to \$D85D - verified before making the change that \$E012 - \$D85D = 1973 bytes exactly matches SECTION 27's real, measured dictionary content, a genuine zero-padding exact fit (was 2048 bytes, 75 bytes of slack). BASECODE moved from \$E800 down to \$E012 to match; its own upper bound was not given and stayed at \$FFBF , growing it from 6080 to 8110 bytes.
- **RESOLVED (since this was written): the three-way collision described below is fully closed.** INOUT moved first (resolving the INOUT third), then RSTACK / DSTACK / CODETOP / APPCODE / APPDICT were all given new addresses (resolving DSTACK / RSTACK), and BASEDICT itself later moved again to \$D83F (from \$D85D ,

shifting down 30 bytes alongside `BASECODE`) as part of closing the separate `INITCODE` / `BASECODE` overlap. A full pairwise sweep of the current memory map (`VECTORS` / `INITCODE` / `BASECODE` / `BASEDICT` / `INOUT` / `RSTACK` / `DSTACK` / `APPCODE` / `APPDICT` / `APPVARS` /all buffer regions/ `GLOBALS`) confirms **zero overlaps anywhere**, re-checked as part of this same update. Kept as history below, not deleted. Original text: `BASEDICT` 's new range (**\$D85D-\$E011**) entirely overlaps three other live regions: `DSTACK` (931 of its 1024 bytes, `$D85D-$DBFF`), `RSTACK` (all 768 bytes, `$DC00-$DEFF`), and `INOUT` (all 256 bytes, `$DF00-$DEFF`). Found by checking the new address against every other region's boundaries before updating the documentation - not caught until then. This is a fundamental, system-breaking conflict, not a byte-budget tightness issue like the `INITCODE` / `BASECODE` overlap noted elsewhere: as configured, the ROM dictionary, the data stack, the return stack, and the ACIA's registers would all be mapped to the same physical addresses simultaneously, which cannot work on real hardware without bank switching (which this system does not have). The requested `EQU` values were applied exactly as given, since that's what was asked; resolving the collision itself was not attempted here, since it requires a decision this response can't make alone - moving `DSTACK` / `RSTACK` / `INOUT` elsewhere, choosing a smaller `BASEDICT` / `BASECODE` split that doesn't reach down this far, or reconsidering whether `$D85D` was the address actually intended. The documented ROM part was still widened from 8K to 16K per the request (real total usage of `BASEDICT` + `BASECODE` + `INITCODE` + `VECTORS` alone, ignoring the collision, is about 10.15K, leaving roughly 6.2K of spare capacity within a 16Kx8 EPROM), but that figure is secondary to the collision above. Verified: zero duplicate symbols, dictionary chain unaffected (219 entries, since this doesn't touch it).

- **The `INITCODE` / `BASECODE` overlap - open since it was first found, mentioned in at least five separate entries above across several turns - is finally resolved.**

`BASECODE` and `BASEDICT` both shifted down exactly 30 bytes (`BASECODE` : `$E012` -> `$DFF4` ; `BASEDICT` : `$D85D` -> `$D83F`), chosen precisely: 30 bytes is exactly the overlap amount, so `BASECODE` 's nominal end (`$FFA1` , unchanged 8110-byte budget) now lands exactly one byte below `INITCODE` 's start (`$FFA2`) - zero gap, zero overlap. `BASEDICT` shifted the same amount to stay perfectly contiguous with `BASECODE` 's new start, preserving its own exact, zero-padding fit. Verified with a full pairwise sweep across every region in the memory map, not just the two that moved: **zero overlaps anywhere** - the first time that's been true since this whole sequence of address changes began. `USROMSTRT` / `USROMEND` containment re-checked and still passes for all three ROM regions. Also cleaned up the accumulated resolved-issue narrative in the top-of-file note (previously ~40 lines chronicling the `BASEDICT` / `DSTACK` / `RSTACK` / `INOUT` / `CODETOP` / `APPDICT` saga turn by turn, including one claim - the `APPDICT` / `APPVARS` overlap being open - that was already stale before this edit) down to a short, current-state summary,

matching the same cleanup already applied to the documentation.

- **forth6809.asm 's four ORG'd blocks physically reordered to match ascending memory address, low to high:** `BASEDICT` , `BASECODE` , `INITCODE` , `VECTORS` (previously `VECTORS` , `INITCODE` , `BASECODE` , `BASEDICT` - roughly the reverse). Pure text reordering only - `ORG` directives set the actual assembled address independent of where in the file each block appears, so this has zero effect on the assembled output; verified rather than assumed by comparing the sorted multiset of every line in the file before and after, which matched exactly (nothing added, removed, or altered - only position changed). This was also done in a freshly-reset environment (the working directory from earlier turns was gone; recovered by copying the last-delivered `forth6809.asm` back from the persistent output location and confirming its hash matched what was last verified). The dictionary chain-walk check initially appeared to fail after reordering (1 of 219 entries reached) - traced to a bug in the verification script itself, not the file: an arbitrary 300-character cap on how much of each header's body text it scanned cut off `ABORTHDR` 's second `FDB` field, since its comment runs several lines longer than most headers. Fixed the script to scan each header's full body instead of a fixed character count, re-ran, and confirmed all 219 entries reachable end to end. Also confirmed zero duplicate symbols and all four region `EQU` s still resolve to their correct, unchanged addresses.
- **ROMSTRT / ROM: / FILL padding block's 256-byte gap is now closed - the ORG target changed from ROMSTRT to USROMSTRT , and ROMSTRT itself moved to the top of the memory-map constants, alongside USROMSTRT / USROMEND rather than sitting alone just before its point of use.** The gap flagged last turn wasn't a formula problem - `BASEDICT- USROMSTRT` (5951 bytes) was already the right byte count, it just needed to start from `USROMSTRT` (`$C100`), not `ROMSTRT` (`$C000`). With `ORG USROMSTRT` , the fill now covers `$C100 - $D83E` exactly, landing with zero gap against `BASEDICT` 's real start (`$D83F`). `ROMSTRT` remains a distinct, real symbol (`$C000` , the true physical EPROM start, still meaningfully different from `USROMSTRT`), now serving purely as a documented reference constant rather than an `ORG` target. `FILL` 's status as an unconfirmed LWASM directive remains open - that wasn't addressed this turn - see the note left in place at the actual `FILL` line. Also fixed a second stale comment found while making these changes: "BASECODE is \$E800" (on the `ORG BASECODE` line, left over from several address changes ago) corrected to `$DFF4`; "BASEDICT is \$E000" was checked and found already correct from a prior turn, needing no change. Verified: zero duplicate symbols, `ROMSTRT` still defined exactly once at its new location, zero remaining `ORG ROMSTRT` anywhere, dictionary chain still 219 entries intact. Also worth noting: this turn's work began by discovering the local working copy had silently diverged from the actually-delivered file (showing `ORG USROMSTRT` when the delivered file actually had `ORG ROMSTRT`) - resolved by re-syncing the working copy from the persistent, hash-

confirmed output file before making any changes, rather than editing from an unverified state.

- **A second `FILL` padding block added, this time between `BASECODE` 's real content and `INITCODE` 's start - same intent and same open verification question as the `ROM: block two turns ago`.** `BASEND`: sits right where `BASECODE` 's actual assembled content ends (the same point `BASECODEEND` marks, immediately before `SECTION 2` begins, now that the file's physical section order has `BASECODE` directly followed by `INITCODE`); `FILL $FF,INITCODE-BASEND` covers whatever gap exists between there and `INITCODE` 's start. Genuinely self-correcting, not a hardcoded guess: because `BASEND` is computed from wherever real content actually ends rather than an estimated figure, this fill's size will automatically be correct once a real assembler runs, regardless of the precision of any manual byte count. Using the precise instruction-by- instruction count from several turns ago (recomputed fresh here to confirm it's still 7984 bytes, unchanged since nothing in the intervening turns touched `BASECODE` 's actual content), this would cover 126 bytes (`$FF24 - $FFA1`) - matching the slack between real content and the nominal 8110-byte budget exactly, as expected. `FILL` 's status as an LWASM directive remains unconfirmed (see the `ROM: block`'s own note) - not re-verified here, since the question is identical for both uses. Verified: zero duplicate symbols, `BASEND` defined exactly once, dictionary chain still 219 entries intact. **RESOLVED - the real listing confirms `FILL` genuinely executes for both blocks; see the dedicated entry earlier in this file (search "FILL's status as a real")**. The 126-byte estimate here is superseded by the real, current figure of 79 bytes, reflecting how much `BASECODE` and the memory map have both moved since this entry was written.
- **Both padding blocks repositioned, and the `ROM: block`'s explanatory comment deleted entirely.** `ORG USROMSTRT / ROM: / FILL` moved from just before `ORG BASEDICT` to just before `SECTION 27` 's comment header instead (now sitting directly after `GLOBALS_USED`); `BASEND: / FILL` moved from just before `ORG INITCODE` to just before `SECTION 2` 's comment header (directly after `BASECODESIZE`). The multi-line "UNVERIFIED: FILL is not a directive..." comment on the `ROM: block` is gone - the underlying uncertainty (whether `FILL` is a real, supported LWASM directive) is unchanged and still real, it's just no longer written down at that specific spot; see the historical entries above for the actual finding. Pure repositioning and deletion, nothing recomputed or changed in substance. Verified: zero duplicate symbols, `ROM: / BASEND:` each still defined exactly once, both blocks confirmed (by text position, not assumption) to sit before their respective section's comment header rather than after, zero remaining "UNVERIFIED" text anywhere, dictionary chain still 219 entries intact. **RESOLVED - the underlying `FILL` uncertainty this note refers to is now settled; see "FILL's status as a real" earlier in this file.**

- MVSCRATCH (the `ORG $0100` block, `MVCNT` / `MVDST` / `MVSRC` and their aliases through `FILLCHR`) moved from right after `GLOBALS EQU $0000` to right after `GLOBALS_USED EQU 256` instead - continuing the same file-order-matches-memory-order cleanup as the `VECTORS` / `INITCODE` / `BASECODE` / `BASEDICT` reordering several turns back.** Moved as one unit: the full explanatory comment block plus all nine lines of actual code, ending exactly at `FILLCHR EQU MVSRC` as specified - `SP0` / `RP0` (unrelated stack- pointer constants that happened to sit right after `MVSCRATCH`) stay in their original position, now following `GLOBALS` directly. Verified with the strongest available check: the sorted multiset of every line in the file, compared against the version delivered immediately before this edit, matched exactly - confirming nothing was added, removed, or altered, only repositioned. Also confirmed the new order is correct (`GLOBALS_USED -> MVSCRATCH 's ORG $0100 -> ORG USROMSTRT` , matching ascending memory address: `$0000 - $00FF` globals, `$0100 - $0105` `MVSCRATCH`, `$C100 + ROM`), zero duplicate symbols, every `MVSCRATCH` -related constant still defined exactly once, dictionary chain still 219 entries intact.
- Conditional assembly added for serial I/O: interrupt-driven (original) and polling (new) implementations now coexist, switched by a single flag, `SERIALPOLL EQU 1` (polling is the default/active mode).** Verified LWASM's actual conditional- assembly directives against the real manual before using them (`IFEQ` / `IF` / `IFNE` / `IFDEF` / `IFNDEF` / `ELSE` / `ENDC` are documented) rather than guessing, unlike the still-unresolved `FILL` question elsewhere. Three conditional blocks, all gated by the same flag: (1) `INFILL` / `RTSCHECKHI` / `RTSCHECKLO` / `IRQH` (the full interrupt-driven receive/transmit/RTS-handshaking logic) - only assembled when `SERIALPOLL=0` ; when `SERIALPOLL=1` , `IRQH` becomes a bare `RTI` stub, matching the other genuinely unused vectors (`NMIH` / `FIRQH` / `SWI2H` / `SWI3H`), since ACIA interrupts are never enabled in polling mode and the vector table unconditionally references `IRQH` by name either way. (2) `KEY` / `KEYQ` / `EMIT` - both versions compile to the same three label names, so the dictionary headers (`H_KEY` / `H_KEYQ` / `H_EMIT`) never need to change regardless of which mode is active. The new polling versions block (`KEY` , `EMIT`) or check once (`KEYQ`) directly against `ACIASR 's` `SR_RDRF` / `SR_TDRE` bits - no ring buffers, no RTS/CTS, fully synchronous. (3) `COLDSTRT 's` ACIA mode-select, now choosing between `CR_RXON` (RX interrupt enabled) and a new `CR_POLL` (`$15 - CR_RXON` with only the RX-interrupt-enable bit cleared; RTS held permanently low, since polling has no ring buffer to overflow and needs no flow control). Checked before writing any of this that no other code references `RTSSTATE` / `INFILL` / `RTSCHECKHI` / `RTSCHECKLO` / `CR_RTSHI` / `CR_RXTX` outside what's already being conditionally wrapped - confirmed clean, nothing left dangling. Verification required a different approach than the usual duplicate-symbol check, since `KEY` / `KEYQ` / `EMIT` / `IRQH` legitimately appear twice in the raw source now (once per branch) - a naive check would false-positive on that. Instead, simulated what

LWASM would actually assemble for each value of `SERIALPOLL` (stripping whichever branch is inactive) and ran the real structural checks against each simulated result separately: both branches independently show zero duplicate symbols, `KEY / KEYQ / EMIT / IRQH` each resolving to exactly one definition, and the dictionary chain fully intact (219 entries) in both cases. Also confirmed the three `IFEQ / ELSE / ENDC` blocks are balanced (3/3/3) in the raw file.

- **SUPERSEDED by a later entry below: moving `ABORTHDR / QUITHDR` into `BASEDICT` reintroduced a 19-byte `BASECODE / BASEDICT` overlap after this was written** - "zero overlaps anywhere" was accurate at the time, not any more. Kept as history, not deleted. Original text: **Current memory map state, re-verified fresh as of this update: zero overlaps anywhere.** A full pairwise sweep across all 18 regions (`VECTORS` , `INITCODE` , `BASECODE` , `BASEDICT` , `INOUT` , `RSTACK` , `DSTACK` , `APPCODE` , `APPDICT` , `APPVARS` , `SIBUF` , `WORDBUF` , `TIBBUF` , `OUTBUF` , `INBUF` , `SERBUF` `idx` , `MVSCRATCH` , `GLOBALS`) found no collisions - every gap/overlap that was ever open (the `INITCODE / BASECODE` 30-byte overlap, the `BASEDICT / DSTACK / RSTACK / INOUT` three-way collision, the `CODETOP / DSTACK` mismatch, the `APPDICT / APPVARS` overlap) has been resolved by earlier turns and stays resolved now, after the `SERIALPOLL` conditional-assembly addition and both `FILL` padding blocks, neither of which touch region boundaries. All three ROM-resident regions (`INITCODE / BASECODE / BASEDICT`) remain genuinely contained within `USROMSTRT..USROMEND` . What remains genuinely open, not a gap/overlap question: whether `BASECODE` 's real assembled size actually fits its 8110-byte nominal budget (the precise manual count is 7984 bytes, still not confirmed by a real assembler - see the earlier follow-up entry), and `FILL` 's status as an actual LWASM directive.
- **`ABORTHDR / QUITHDR / BASELATEST` moved from their own separate section into `BASEDICT` proper, at the end of the dictionary entries (right after `DICTTOP`), and renamed to `H_ABORT / H_QUIT` to match this file's `H_` naming convention.** This wasn't purely cosmetic: these two headers were previously sitting physically inside `BASECODE` 's address range (Section 26 sits before `ORG BASEDICT` in the file), even though they're header *data*, not code - meaning their bytes were being counted against `BASECODE` 's budget instead of `BASEDICT` 's. Moving them into the actual `ORG BASEDICT ... BASEDICTEND` block fixes that miscounting, at the cost of a new, real consequence: `BASEDICT` 's real size grew from 1973 to 1992 bytes (the 19 bytes `H_ABORT` and `H_QUIT` actually occupy), but `BASECODE` 's start (`$DFF4`) is a fixed address that doesn't move just because `BASEDICT` 's real content grew - reintroducing a genuine 19-byte overlap between them (`$DFF4-$E006`), the same class of problem as the original `INITCODE / BASECODE` overlap resolved several turns ago. A full pairwise sweep of the entire memory map confirms this is the *only* new overlap - nothing else is affected. Not

fixed here, since it needs a decision (shift `BASECODE` by 19 bytes to match, the same kind of fix used last time; or something else) rather than a mechanical follow-up.

`TRUEBODY` / `FALSEBODY` (real code, correctly retitled) stayed in `BASECODE` where they belong - only the two header structures moved. Verified: zero duplicate symbols (checked against both `SERIALPOLL` branches via the same simulation-based method established for that feature), dictionary chain still 219 entries, now correctly walked starting from `H_QUIT` rather than `QUITHDR`, ending cleanly at 0 in both configurations. Also fixed two separately-stale items caught while making this edit: the Section 27 header comment still cited `BASEDICT` as `$D85D-$E011` (from before the 30-byte shift several turns back) and the top-of-file summary's "zero overlaps anywhere" claim, both now updated to reflect this change and its real consequence.

- **HISTORICAL, superseded - `BASECODE` has since moved several more times; the current, real state (confirmed via `forth6809_1st.txt`) has zero overlap anywhere in the memory map, not the 19-byte overlap this entry describes. Kept as history, not deleted, since it was a genuine finding at the time.** `BASECODE` shifted up 19 bytes (`$DFF4` -> `$E007`) to close the `BASECODE` / `BASEDICT` overlap from last turn - but the overlap wasn't eliminated, it was relocated. `BASECODE` 's new start does land exactly one byte above `BASEDICT` 's real end (`$E006`), closing that specific 19-byte overlap precisely. But `BASECODE` 's nominal size (8110 bytes) didn't change, so its nominal end shifted by the same 19 bytes too - from `$FFA1` to `$FFB4`, which is now 19 bytes *into* `INITCODE` 's start (`$FFA2`). A full pairwise sweep of the entire memory map confirms exactly one overlap remains, same total byte count (19), just at a different boundary (`INITCODE` / `BASECODE` instead of `BASECODE` / `BASEDICT`). This was checked and reported precisely, not assumed away or glossed over: shifting only `BASECODE` 's *start* while its budget stays fixed cannot reduce total overlap when that budget was already calibrated (in an earlier turn) to exactly reach `INITCODE` 's start from the *old* position - moving the start without shrinking the budget just pushes the same problem to the other end. Actually eliminating both overlaps at once would need `BASECODE` 's nominal size reduced by 19 bytes (to 8091), not just its start address moved; not done here, since that's a different change than what was asked. Verified: zero duplicate symbols in both `SERIALPOLL` branches (same simulation method as before), dictionary chain still 219 entries in both configurations. Also fixed a real arithmetic error caught before delivery: an initial draft of this comment miscalculated the new `INITCODE` overlap as 7 bytes at `$FFAE` - corrected to the actual 19 bytes at `$FFB4` before this was ever shipped.
- **DICTTOP confirmed genuinely unused and removed, along with the one comment that named it.** Checked precisely before touching anything: defined once (`DICTTOP EQU H_FALSE`), and referenced exactly one other place in the whole file - inside a prose comment describing the dictionary chain, not by any real code. No `EQU` depended on it,

nothing JSR 'd or FDB 'd it. Confirms the suspicion that raised this: BASELATEST (EQU H_QUIT , the true overall chain head used by COLD to initialize LATEST) is what's actually used at runtime; DICTTOP was a separate, distinct value (H_FALSE , the newest of the 217 generation-pass entries specifically, not the true head) that nothing ever consumed. Removed the EQU line and rewrote the one comment that named it to describe H_FALSE directly instead. Verified: zero duplicate symbols and dictionary chain still 219 entries intact in both SERIALPOLL branches (same simulation method as the last two turns).

- Serial init code (LDA #\$03 through STA ACIACR , including the SERIALPOLL mode-select) extracted from COLDSTRT into a new subroutine, INITSERIAL , and moved into the ACIA Interrupt section (SECTION 3), right before the IFEQ SERIALPOLL that gates INFILL .** COLDSTRT now just does JSR INITSERIAL where the inline code used to sit. The extracted block keeps its internal SERIALPOLL conditional (CR_RXON vs CR_POLL) exactly as it was, just relocated and given an RTS . Small, bounded side effect worth naming rather than silently ignoring: this shifts roughly 10 bytes of real content from INITCODE to BASECODE (the subroutine itself, plus the shrink from removing the inline code and adding a 3-byte JSR in its place) - well within the existing, already-acknowledged estimation uncertainty for both regions, not something that changes the tracked overlap numbers meaningfully. Verified: IFEQ / ELSE / ENDC still balanced (3/3/3 - moving a conditional block doesn't add a new one), INITSERIAL defined exactly once and correctly resolved by its JSR , zero duplicate symbols and dictionary chain still 219 entries intact in both SERIALPOLL branches (same simulation method as the last several turns).
- Real bug fixed: INTERPRET called WORD without pushing a delimiter character first, unlike every other caller of WORD in this file.** Found while tracing a full character-by-character simulation of the ACIA receiving "ABC"+CR under polling mode. WORD 's calling convention (PULU D / STB DELIM) requires its caller to push a delimiter char first - confirmed by checking all 7 call sites: HEADER , TOW , ISW , SQUOTE , CHARW , and LPAREN all correctly push one (LDD #32/#34/#') ' then PSHU D) immediately before JSR WORD . INTERPRET was the only one that didn't. Traced the actual consequence: QUIT invokes the interpreter via CATCH , which only pulls the xt off U and pushes nothing back (its bookkeeping goes on S , the return stack, not U) - so WORD 's stray PULU D reads from an empty U stack. SP0 (U 's empty/reset value) is exactly RSTACK 's first byte, not unused margin, so DELIM ended up holding live return-stack content instead of a real delimiter - unpredictable, not necessarily space, meaning WORD couldn't reliably find token boundaries at all. Confirmed this is not polling-specific, per instruction: IRQH (the interrupt-driven receive path) never touches U at all, so interrupt-driven mode had no incidental workaround either - same bug, both branches. Fixed by pushing LDD #32 / PSHU D right at the ILOOP: label before JSR WORD , matching every other call site's

convention exactly rather than inventing a different approach - this single insertion covers every iteration of the loop, since every branch back to `ILOOP` (from `DOEXEC`, `CCALL`, `TRYNUM` 's success path) re-enters at this exact point. Verified: zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches (same simulation method as recent turns).

- **CORRECTED the same turn this was written, by real assembler data: the “new VECTORS overlap” claimed below did not actually exist - it was computed from a 78-byte manual estimate that turned out to be wrong by 7 bytes.** A real assembler run reports `INITCODE` 's actual size as 71 bytes (`$47`), not 78. At `$FFA9`, 71 real bytes end at exactly `$FFEF` - one byte below `VECTORS`, zero gap, zero overlap. The `BASECODE` overlap described below (12 bytes against its nominal budget) is unaffected by this correction and remains open. Kept as history, not deleted - this was a real, reasoned finding at the time, just based on the best information available before a real assembler had actually been run against this specific figure. Original text: `INITCODE` **shifted up 7 bytes (`$FFA2` -> `$FFA9`) - reduces one overlap but introduces a different, new one.** Computed precisely before and after: against `BASECODE` 's nominal 8110-byte budget, the overlap shrinks from 19 bytes to 12 (`$FFA9` - `$FFB4`) - real progress, not a fix. But `INITCODE` 's real content (78 bytes, an established figure from `COLDSTRT + WARM + WARMMSG`, not an estimate) used to end at `$FFEF`, exactly one byte below `VECTORS` ' start (`$FFF0`) - zero gap. At `$FFA9` it now ends at `$FFF6`, 7 bytes *into* `VECTORS` ' own territory - a brand new overlap that didn't exist before this change, and arguably more certain than the `BASECODE` one, since it's based on solid content rather than a budget estimate. Worth noting: checked what happens using `BASECODE` 's *real* 7984-byte content instead of its nominal budget - the `INITCODE` / `BASECODE` overlap disappears entirely under that assumption, leaving only the new `VECTORS` overlap. Applied exactly as requested; neither overlap resolved here. Verified: zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches.
- **The real, assembler-confirmed figure: `INITCODE` is 71 bytes (`$47`), told directly rather than derived from a manual count - the first genuine assembler-reported figure in this whole project, as opposed to every prior byte count in this file, which has been a manual estimate with an explicitly acknowledged margin of error.** Corrected the one place in `forth6809.asm` that cited the old, wrong 78-byte figure (`INITCODE` 's own `EQU` comment), and the checklist entries above. Left two older, already-properly-hedged entries alone rather than edit them - they already said “still an estimate... not a real assembler's output” at the time, so nothing in them is actually wrong, just superseded. `BASECODE` 's real size (7984 bytes, from the instruction-by-instruction manual count) remains unconfirmed by an actual assembler run - this correction applies specifically to `INITCODE`, not the other regions' still-estimated

figures. **CONFIRMED** - `forth6809_1st.txt` 's own `INITSIZE` computes to exactly \$47 /71 bytes, matching this figure precisely. `BASECODE` 's real size is now also known (8304 bytes) - see the "DEFINITIVE, real figures" entry earlier in this file.

- **RESOLVED (since this was written) - confirmed via the real listing**
(`forth6809_1st.txt`): the current `TRUEBODY` / `FALSEBODY` show the correct, standard convention (`TRUEBODY: LDD #$FFFF` , `FALSEBODY: LDD #$0000`), matching `TRUEV` / `FALSEV` and every comparison operator. The inversion this item flagged was fixed at some later point in the session, but this checklist item itself was never updated to reflect it - a real gap, not a false alarm, caught during a systematic re-check of every remaining open item. `TRUE` and `FALSE` replaced with simple subroutines, per explicit request - but the requested values are inverted from the rest of the system's true/false convention, and this was applied exactly as asked, not corrected. `TRUEBODY` is now `LDD #$0000 / PSHU D / RTS` ; `FALSEBODY` is now `LDD #$FFFF / PSHU D / RTS` - both simple, direct subroutines (matching the style of e.g. `BLW`), replacing the `DODOES`-trampoline `CONSTANT`-pattern implementation they used before (`TRUEVAL` / `FALSEVAL` , the value cells that pattern needed, removed along with it - confirmed unreferenced anywhere else first). Flagging this prominently rather than as a routine note: `TRUEV EQU $FFFF` / `FALSEV EQU $0000` , defined near `GLOBALS` , are what every comparison operator in this system (`=` , `<` , `>` , and the rest) actually pushes for a true/false result - that convention is unchanged. `TRUE` and `FALSE` , the two words, now push the opposite of what those operators mean by "true" and "false." Any code that does something like `TRUE IF ... THEN` would behave backwards from what every comparison-based conditional in the same program does, since `0` is false-for-branching on this CPU regardless of which named constant produced it. Verified: zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches; confirmed `DODOES` and `ATSIGN` remain genuinely used elsewhere (by `CREATE` / `CONSTANT` / `VARIABLE` 's own runtime code) and were not left orphaned by this change.
- **`TRUE` / `FALSE` corrected to the standard convention: `TRUE` pushes `$FFFF` , `FALSE` pushes `$0000` - each a direct `LDD #value / PSHU D / RTS` , no indirection.** The code found at the start of this turn had these inverted (`TRUEBODY` loaded `$0000` , `FALSEBODY` loaded `$FFFF`), with a comment explicitly documenting that inversion as deliberate, from an earlier request not captured in this checklist - likely predating a context boundary, since no corresponding entry exists above to mark as superseded. That comment also correctly flagged the consequence at the time: `TRUE` / `FALSE` disagreed with `TRUEV` / `FALSEV` (`$FFFF` / `$0000`) and every comparison operator (`=` , `<` , `>` , and the rest), which all return `TRUEV` for true and `FALSEV` for false. This turn's request restores the standard values, resolving that inconsistency. `TRUEVAL` / `FALSEVAL` (the old

DODOES-trampoline value cells) were already removed in the prior change - nothing further to clean up there. Verified: `H_TRUE / H_FALSE` 's `FDB TRUEBODY / FDB FALSEBODY` CFA fields unaffected (only the pushed values changed, not the labels), zero duplicate symbols, dictionary chain still 219 entries intact in both `SERIALPOLL` branches.

- **FIND confirmed genuinely missing a dictionary entry - added.** Checked first, not assumed: searched for any `H_FIND` label or `FCC "FIND"` string anywhere in the file, found neither. Before adding a header, verified `FIND` 's actual implementation against the standard ANS stack effect (`c-addr -- c-addr 0 | xt 1 | xt -1`) rather than assuming it matched: traced the success path (`FMATCH / FPUSH`) - pushes the `xt`, then checks the header's bit 7 flag, pushing `1` if set or `-1` (via `FISNORM`) if clear, matching immediate-vs-normal exactly - and the failure path (`NOTFOUND`) - reconstructs the original `c-addr` (`SNAMEP-1`) and pushes it alongside `0` . Both match the standard exactly; no code changes needed, only the header. Also added four simple number words - `1` , `-1` , `2` , `-2` - same pattern as `TRUE / FALSE : direct LDD #value / PSFU D / RTS` , no indirection. Code labels `ONEBODY / MONEBODY / TWOBODY / MTWOBODY` , since `1 / -1 / 2 / -2` aren't valid 6809 assembler labels. All five new headers (`H_FIND` , `H_1` , `H_M1` , `H_2` , `H_M2`) chained after `H_QUIT` , with `H_M2` now the true chain head - `BASELATEST` updated accordingly (still referenced symbolically by `COLD` , so no code change needed there either). Updated the two comments that named `H_QUIT` as the head to reflect the new one. Verified: zero duplicate symbols, dictionary chain now 224 entries (was 219), correctly ordered and ending at `0` , in both `SERIALPOLL` branches; byte-exact split-file reassembly.
- **HISTORICAL, superseded - BASECODE has since moved several more times; the current, real state (confirmed via `forth6809_1st.txt`) has zero overlap and zero gap on both boundaries this entry describes. Kept as history.** `BASECODE` shifted up 35 bytes (`$E007 -> $E02A`) - two effects, neither an improvement. (1) Opens a new 35-byte gap between `BASEDICT` 's real end (`$E006` , 1992 bytes) and `BASECODE` 's new start - previously exactly contiguous, zero gap. Not itself broken (a gap isn't a collision), just unused address space where there wasn't any before. (2) Worsens the existing `BASECODE / INITCODE` overlap: `BASECODE` 's nominal size (8110 bytes) is unchanged, so its nominal end shifted up the same 35 bytes too, from `$FFB4` to `$FFD7` - against `INITCODE` (`$FFA9`), the overlap grows from 12 bytes to 47. Full pairwise sweep confirms this is the only overlap anywhere in the memory map, just larger than before. Applied exactly as requested; neither effect resolved here. Also fixed two separately-stale comments caught while making this change: the `ORG BASECODE` line still cited `$DFF4` (two shifts behind), and `BASEDICT` 's own comment still claimed `BASECODE` "moved again to match," which is no longer true now that a gap exists between them. Verified: zero duplicate symbols and dictionary chain still 224 entries intact in both `SERIALPOLL`

branches; byte-exact split-file reassembly.

- **Real bug, found via actual MAME debugger single-stepping (not static analysis) and fixed:** `WORD` 's `EMPTY` branch pushes a c-addr (`WORDBUF` 's address, matching its normal contract) that `INTERPRET` never consumed when the parsed token was empty. `INTERPRET` 's check after `JSR WORD` was `LDX ,U` (a peek, not a pop) / `LDA ,X` / `BEQ IDONE` , and `IDONE` was a bare `RTS` - meaning on every line, once nothing remains to parse, the final `WORD` call's returned address gets stranded on `U` instead of being consumed the way the `JSR FIND` path does (via `FIND` 's own `PULU X`). This happens on every line, including successful single-word ones - it's just invisible there since nothing inspects the stack afterward. On lines with more content, the stranded address sits underneath legitimate values and accumulates across lines with nothing ever cleaning it up. Diagnosed collaboratively: static tracing of `WORD` / `FIND` / `EXECUTE` / `DOT` 's entire numeric chain across several turns kept coming back clean (correctly - none of them had the bug), before an actual MAME debugger session found a spurious address sitting on top of an otherwise-correctly-pushed value, which pointed straight at this exact mechanism once checked against the code. Fixed by changing `IDONE` to `PULU X` / `RTS` , matching the same convention `FIND` already uses to consume a c-addr, rather than inventing a different fix. `IDONE` is referenced from exactly one place, so the fix at the label covers the only path that reaches it; confirmed `CATCH` (the caller once `INTERPRET` returns) doesn't rely on `X` at that point either. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.
- **Real, serious bug found via MAME debugger and fixed:** `WORD` stored the wrong length byte for every token followed by more input on the same line - `TFR X,D` (part of the `TOIN` calculation) silently destroyed `B` , which is `D` 's low byte and also where `SCANLP` 's own `INCB` loop had been counting the token's true character count. `STB ,X+` then stored `TOIN` 's delta instead of that true count - one too many, since the delta includes the consumed trailing delimiter. The copy loop then copied one byte past the token's real end, corrupting every multi-token line's first N-1 tokens (the last token on a line is terminated by running out of buffer, not by `CONSUME` , so its `TOIN` delta happens to equal its true length - which is exactly why every single-token-line test earlier in this conversation, including my own manual traces, never surfaced this). This also means my own earlier traces of this exact routine, across several turns, were themselves quietly wrong in this specific respect - not because the routine's overall logic was mis-modeled, but because I treated `B` and `D` as if they could independently hold different values, when `B` is `D` 's low byte and any write to `D` clobbers it. Diagnosed collaboratively: found via actual MAME debugger single-stepping, not static tracing - confirmed against the source once precisely described. Fixed by saving `B` (`PSHS B`)

before the `TFR X,D / SUBD SRCADDR / STD TOIN` sequence and restoring it (`PULS B`) immediately after, right before `STB ,X+` - placed at the `ENDW` label itself, which both paths that reach it (`CONSUME` 's fall-through and `SCANLP` 's direct branch when the buffer runs out) correctly pass through. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.

- Real, serious bug found via MAME debugger and fixed: `CCALL` (compiles a `JSR` to a given `xt` - used by every colon definition to compile a call to each word it contains) corrupted the target address on every single call.** `PULU D` pulled the `xt` first; `LDA #OPJSR` (loading the opcode to compile) then overwrote `A` - which is `D` 's high byte, so this silently destroyed the top byte of the address just pulled. `STA ,X+` correctly wrote the opcode (A happened to hold the right value for that specific write), but `STD ,X++` then wrote the corrupted `D` out as the call target - a `JSR` to a garbage address, one wrong byte away from the real target. This affects every colon definition unconditionally, not an edge case - `: tv . ;` failed exactly this way, confirmed via MAME debugger as "attempted execution of random memory." Fixed by deferring `PULU D` until after `STA ,X+ : LDX CODEHERE / LDA #OPJSR / STA ,X+ / PULU D / STD ,X++ / STX CODEHERE / RTS - D` is never live at the same time `A` gets reused for the opcode, so nothing clobbers it. The external calling convention (caller pushes the target address, then `JSR CCALL`) is unchanged - only the internal instruction order moved, confirmed by checking a caller still correctly pushes the address before the call. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.
- Real bug found via MAME debugger and fixed: `CONSTANT` compiled a self-referential PFA field instead of one pointing at the actual value cell.** `LDD CODEHERE` read `CODEHERE` 's value *before* the `JSR CODECOMMA` that writes the PFA field itself - at that point `CODEHERE` is the address of that very cell, not of the value 2 bytes further on where the subsequent `JSR COMMA` appends the real number. The compiled structure ended up as `[JSR DODOES][FDB ATSIGN][FDB <address of itself>][value] - DODOES / @` correctly fetches through the PFA, but the PFA pointed at itself, so executing the constant returned its own compile-time address instead of the stored value. `1234 CONSTANT c1` then executing `c1` returned the `CODEHERE` address, not `1234` , exactly matching the debugger trace. Checked `VALUEW` (the structurally similar routine right below `CONSTANT`) for the same issue and confirmed it's genuinely different, not just superficially similar - its PFA points into `VARHERE` (a separate mutable region), and the value is stored at that same `VARHERE` position via `VCOMMA` , so no self- reference exists there; left unchanged. Fixed `CONSTANT` by adding `ADDD #2` after `LDD CODEHERE` , accounting for the PFA field's own 2-byte width so it correctly lands on the value cell that follows rather than on itself. Verified: zero duplicate symbols, dictionary chain still

224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.

- **Real bug found via MAME debugger and fixed in three places: `DOTEST` (the `LOOP` runtime), `DOPLUSTEST` (the `+LOOP` runtime), and `LEAVE` all popped the return address off `S (PULS X)` before finishing their fixed-offset reads/writes of the loop-control cells `DOSETUP` had pushed - shifting every subsequent `2,S / 4,S / 6,S` access by 2 bytes.** Confirmed the exact layout `DOSETUP` pushes (`0,S` =return addr, `2,S` =index, `4,S` =limit, `6,S` =leave-flag) and that those specific offsets were correctly designed for that unshifted layout - the only bug was the premature pop happening before they were read. : lpy 10 3 DO I . LOOP ; failed exactly this way: `DOTEST` 's `LDD 6,S` (meant to check the leave-flag) instead read straight through into whatever return address was already on `S` below the loop cells (`EXECUTE` 's own, in this case) - almost always nonzero, so `BNE DTEXT` fired on the very first pass, exiting the loop immediately. Fixed all three by deferring `PULS X` until each path (continue vs. exit) actually needs it, rather than popping once up front - `DOTEST` / `DOPLUSTEST` each need it in two separate places (the normal-continue path and the exit/ `DTEXT` / `DPTTEXT` path), so it's deferred separately in each rather than moved to one shared spot. Checked `?DO` for a separate copy of this bug and confirmed it reuses the same, now-fixed `DOTEST` via `LOOP` 's own compilation - no separate fix needed there. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.
- **Real, systemic bug found via MAME debugger and fixed in seven places: `LOOP` , `+LOOP` , `UNTIL` , `AGAIN` , `REPEAT` , `ELSE` , and `ENDOF` all saved a branch target in `X (PULU X)` , then called `CCALL` and/or `CODECOMMA` before using it - both of which internally reload `X (LDX CODEHERE / LDX #CODEHERE via APPENDCELL)` , silently destroying the saved target before it was ever used.** Found while confirming a specific report on `LOOP` : : lpy 10 3 DO I . LOOP ; compiled a branch displacement of `+8` instead of `-8` , since `TFR X,D` read the address of the `CODEHERE` variable itself (left there by `CODECOMMA` 's internal `APPENDCELL`) instead of the real loop-body target `PATCH` needed - `DOTEST` then returned to a near-zero, invalid address. This had been masked until the `DOTEST` return-stack-offset bug (an earlier turn this session) was fixed - the loop never previously executed far enough to reach this code path. Once confirmed on `LOOP` , swept every other `PULU X` in the file rather than fix only the reported case: found the identical pattern in `+LOOP` (`DOPLUSTEST` 's compile side), `UNTIL` , `AGAIN` , `REPEAT` , `ELSE` , and `ENDOF` - seven total. Two different fixes depending on structure: where nothing else touched `U` between the pop and the eventual use (`LOOP` , `+LOOP` , `UNTIL` , `AGAIN` , `REPEAT`), parked the target on `U` itself (immune to `CCALL` / `CODECOMMA` , which only touch `D / X / A`) and retrieved it via a later `PULU D` in place of `TFR X,D` . Where something else (`CODEHERE`) got pushed onto `U` in between

(ELSE , ENDOF), parking on U would have retrieved the wrong value - used the MSCR scratch variable instead. Caught this distinction the hard way: an initial attempt at the ELSE / ENDOF fix incorrectly reused the park-on- U approach and would have retrieved CODEHERE instead of the saved target; caught and corrected before delivery by re-tracing the exact U sequence rather than assuming the same fix pattern applied uniformly. Verified every other PULU X in the file individually and confirmed none of the remainder are vulnerable - either no CCALL / CODECOMMA call sits between the pop and the use (ECPATCH , THEN , the ?DO -forward-patch tails in LOOP / +LOOP , ISFOUND / AOFFOUND), or the routine doesn't compile code at all (the plain runtime memory/stack words). Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both SERIALPOLL branches; byte-exact split-file reassembly.

- Real bug found via MAME debugger and fixed: IWORD (I) and JWORD (J) each bracketed a fixed-offset return-stack read with an unnecessary PULS X / PSHS X pair, shifting S by 2 before the read - so I returned the loop limit instead of the index, on every iteration.** : lpy 10 3 DO I . LOOP ; now runs the correct number of iterations (the DOTEST/register- clobbering fixes from earlier this session), but printed the limit (10) ten times instead of the index (3..9). Traced against the layout established while fixing DOTEST : DOSETUP pushes [return-addr@0,S][index@2,S][limit@4,S][leave-flag@6,S] , and 2,S unshifted already correctly targets the index - the PULS X / PSHS X pair served no purpose (nothing needed offset 0 for anything in either routine) and only shifted every subsequent offset by 2, landing one field over. JWORD had the identical bug at its own offset (10,S , correct for the outer loop's index in a nested DOSETUP layout, but wrong once shifted). Fixed both by removing the pop/push pair entirely, per the suggested fix, leaving the existing offsets (2,S , 10,S) unchanged since they were already correct for the unshifted stack. Swept every other PULS X in the file for the same bracketing pattern before concluding the fix was complete: checked UNLOOP and EXITUNLOOP specifically, since both are directly related to loop-frame teardown - confirmed both are structurally different and correct as-is, since they discard *counted blocks* of bytes (LEAS 6,S / LEAS 8,S in a loop) rather than reading a value at one fixed offset, so they're unaffected by any temporary shift regardless. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both SERIALPOLL branches; byte-exact split-file reassembly.